Module 5

Discipline for Trustworthy AI Output

50 minutes

The hour

- 5 min — framing
- 18 min — three concepts (spec-first, golden rule, bidirectional sync)
- 22 min — review a PR against its spec
- 5 min — debrief

10-minute break after.

The gap

You already write a lot of the code using AI.

You already do AI-assisted PR review.

But few teams have a discipline for drift.

> AI generates code. The code drifts from intent.
>
> Your AI reviewer says "looks fine."
>
> AI confidently reviewing AI confidently wrong.

AI speed isn't a contest of model IQ —

it's a contest of how clearly you can frame what you want.

*Module 3 was trust **across teammates**. Module 4 was trust **in delegation**. This is the one where you earn trust **in the code itself**.*

Spec-first

Before code, write down what you're trying to do.

- Acceptance criteria
- Constraints
- Out of scope

The spec is the durable artifact. Everything you generate is measured against it.

Defends against: **discarded specs**, **one-shot generation**.

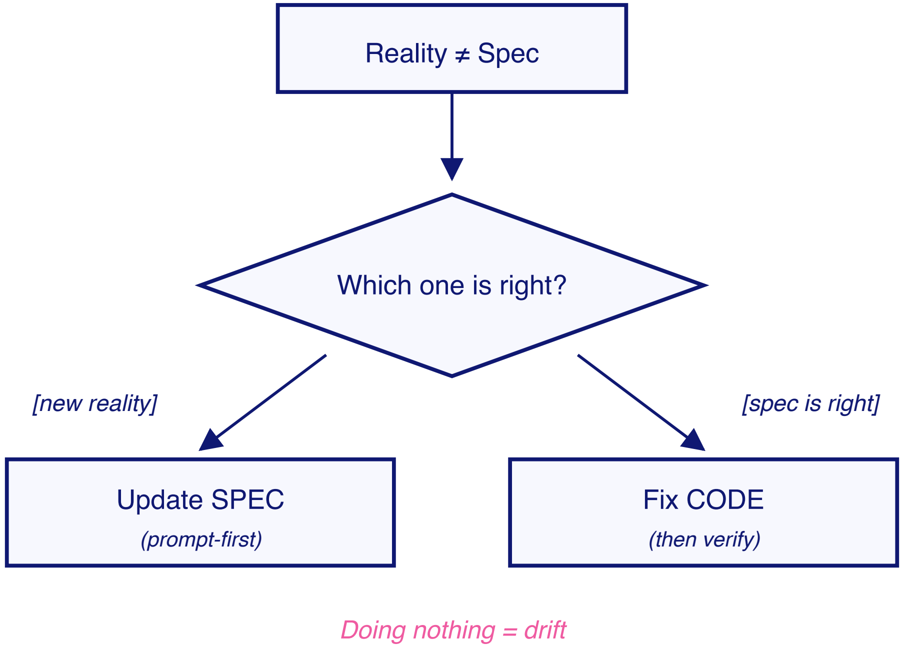
Some teams call this a "plan". Same artifact, more throwaway-sounding name.

Why a *versioned* spec pays off

- **Auditability** — *"why did we build it that way?"* answered from the repo, not memory.
- **Onboarding** — spec + code recover intent. Code alone hides *why*.
- **The anchor for sync** — only a versioned, reviewable spec can be *diffed* against the code. Without it, "is the spec wrong or is the code wrong?" has no answer.

The version isn't ceremony. It's what makes the next two concepts work.

The golden rule



When reality diverges from the spec,

fix the prompt first.

Defends against:

code-first divergence.

Golden rule in practice

A teammate adds a `fromDate` filter to `list_routes`.

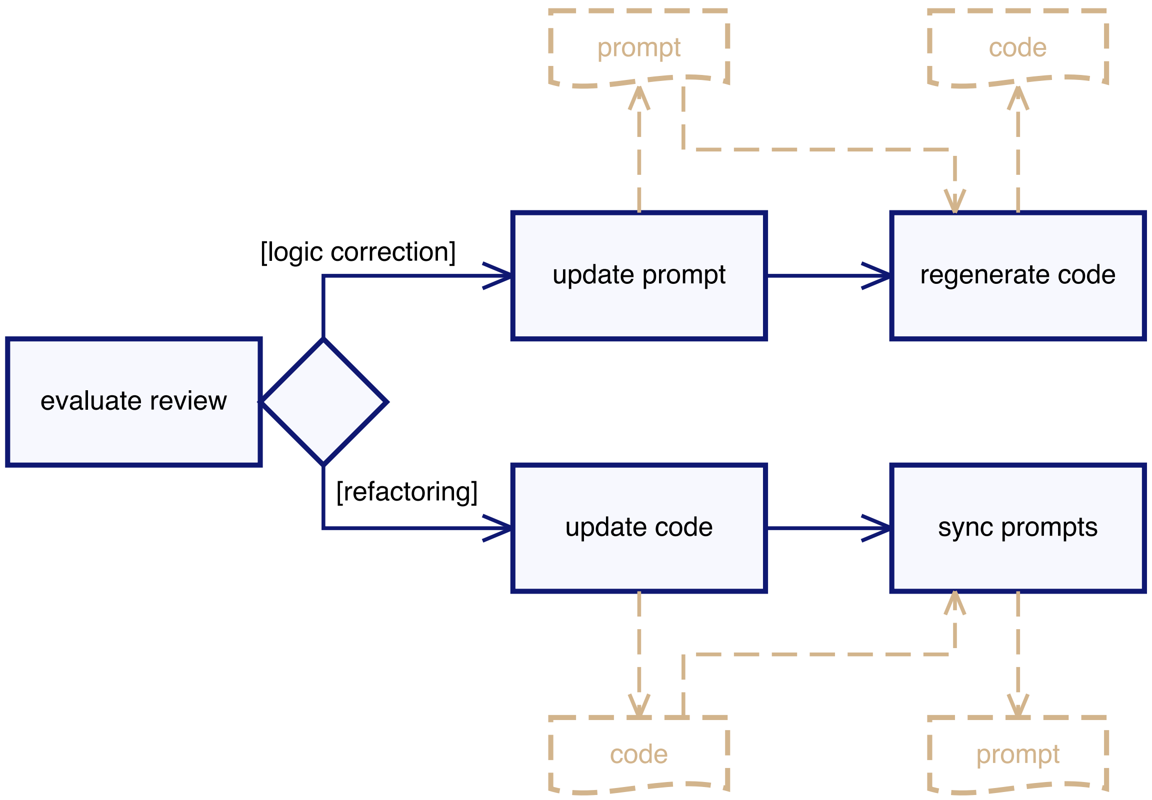
The spec marks date-range filters **out of scope** — explicit.
The PR description calls it "a useful addition."

Six weeks later: timezone bug in the filter. Nobody remembers why it shipped. The spec said no; the code says yes; source of truth is ambiguous.

> "Helpful addition" without updating the spec ≠ helpful.
> Either spec is wrong (update it in this PR) — or code is wrong (remove the filter).
> **Pick one, in writing. Doing nothing is the failure.**

This is exactly the exercise you'll do shortly.

Bidirectional sync



Two legitimate update flows.

Anything else is drift.

Defends against:

wholesale regeneration.

Bidirectional sync in practice

Logic correction. Acceptance criterion says routes return ordered by `scheduledStartAt`. Code returns insertion order. Team confirms ordering matters for the consumer. → Spec is right; code is wrong. Fix code, regenerate, verify ordering test.

Refactor. Handler has a magic `300_000` for a timeout. Pull it to `ROUTE_TIMEOUT_MS`. Acceptance criteria don't change — externally-observable behavior is identical. But the spec's *implementation notes* referenced the old inline number. → Sync back: update the notes, commit.

> Change *size* matches divergence *size*. Refactor doesn't trigger regeneration.

Discipline as code: hooks

The first three are disciplines you have to *remember*.

Hooks are disciplines you *encode*.

Configure in `.claude/settings.json`. Agent has to obey.

- **`PostToolUse`** on Edit/Write → run prettier
  Sloppy edits never ship.
- **`PreToolUse`** on Bash → reject `rm -rf`, `--force`
  Don't trust the agent to be careful.
- **`UserPromptSubmit`** → append prompt + timestamp to a log
  Free audit trail.

Intent vs **enforcement**.

The exercise

A teammate opened a PR for `list_routes`.
You all spec'd it together last sprint.

Read through these:

- `modules/module-5/sample/spec.md`
- `modules/module-5/sample/pr-description.md`
- `modules/module-5/sample/typescript/proposed/src/tools/list-routes.ts`

Would you approve it?

Open Claude. Compare spec vs code with it. Don't review in isolation.

Reviewing — what to do

For each divergence you find:

1. **Identify** it precisely. Which line? Which acceptance criterion?
2. **Decide** who wins — the spec or the code? Why?
3. **Decide** what action follows — change the code, or update the spec?

Push back on Claude's first answer. Walk every acceptance criterion.

Coming up: Module 6

Discipline scales into automation.

AI agents in pipelines that triage failures before a human opens the logs.

10-minute break.